

System Design

Java Backend Interview — Short Notes

1. System Design Interview Approach (Framework)

- **Step 1 — Clarify Requirements (5 min):** functional (what it does), non-functional (scale, latency, consistency, availability)
 - Users: how many (DAU/MAU)? reads vs writes ratio? data size? geography?
 - SLA: latency p99? availability (99.9% vs 99.99%)? durability?
- **Step 2 — Back-of-envelope estimation (3 min):** QPS, storage, bandwidth, memory needs
- **Step 3 — High-level design (10 min):** draw boxes — clients, LB, services, DB, cache, CDN, message queue
- **Step 4 — Deep Dive (15 min):** bottlenecks, data model, API design, scalability, tradeoffs
- **Step 5 — Wrap-up:** failure scenarios, monitoring, future scale

1.1 Estimation Cheat Sheet

Metric	Estimate	Metric	Estimate
Read QPS (social media)	10K–100K req/s	1 day seconds	~86,400 s
Write ratio	Read:Write usually 100:1	1 month seconds	~2.5M s
Avg tweet/post size	~500 bytes	Avg user photo	~300 KB
Avg video (4K)	~8 GB/hr	1 TB storage	~3M photos
L1 cache latency	~0.5 ns	RAM latency	~100 ns
SSD random read	~100 µs	HDD seek	~10 ms
Network round trip	~1 ms (same DC)	Cross-region	~150 ms

2. Scalability Fundamentals

2.1 Vertical vs Horizontal Scaling

	Vertical (Scale Up)	Horizontal (Scale Out)
Approach	Add CPU/RAM/SSD to same machine	Add more machines behind load balancer
Limit	Hardware ceiling — most expensive machines have limit	Theoretically unlimited with good architecture
Fault tolerance	Single point of failure	Redundant — one node down = others serve
Complexity	Simple — no code change	Stateless design required, distributed coordination
Cost	Expensive at high end	Cost-effective with commodity hardware
Use for	DBs in early stage, quick fix, stateful workloads	Web/API servers, microservices, read replicas

2.2 Load Balancing

Algorithm	How	Best For
-----------	-----	----------

Round Robin	Cycle through servers in order	Equal-capacity servers, stateless
Weighted Round Robin	More requests to higher-capacity servers	Mixed server capacities
Least Connections	Route to server with fewest active connections	Long-lived connections (WebSocket)
IP Hash	hash(client IP) % servers — sticky sessions	Session affinity needed without server-side session storage
Consistent Hashing	Server ring — minimal remapping when servers added/removed	Distributed cache, stateful routing
Random	Random server selection	Simple, no tracking overhead

- Layer 4 LB (TCP/UDP): fast, no content inspection. Layer 7 LB (HTTP): route by URL, headers, cookies. Use L7 for microservices

2.3 CAP Theorem

- In network partitions, choose between **Consistency** (all nodes see same data) OR **Availability** (every request gets response)

Type	Description	Examples	Use When
CP	Consistent + Partition Tolerant. May reject requests during partition	HBase, Zookeeper, etcd, MongoDB (single-master)	Financial data, inventory, counters
AP	Available + Partition Tolerant. May return stale data	Cassandra, CouchDB, DynamoDB (eventual)	Social feeds, recommendations, DNS
CA	Consistent + Available. Only single node — no partition tolerance	RDBMS (traditional), SQLite	Single-server only

- ✓ **PACELC** extends CAP: even without partition (E), tradeoff between Latency and Consistency. Most practical systems are PACELC

3. Caching

3.1 Cache Strategies

Strategy	Flow	Pros/Cons	Use For
Cache-aside (Lazy)	App: check cache → miss → read DB → populate cache	+ Simple, resilient. – Cache miss on cold start, thundering herd	General purpose, read-heavy. Most common
Read-through	Cache sits in front of DB — on miss cache fetches from DB	+ App always talks to cache. – Implementation complexity	ORM-integrated caches (Hibernate L2)
Write-through	Write to cache + DB synchronously	+ Cache always fresh. – Write latency (2 writes), cache bloat	Read-after-write consistency needed
Write-behind (async)	Write to cache → async write to DB	+ Low write latency. – Data loss if cache fails before DB write	High write throughput, eventual consistency OK
Write-around	Write directly to DB, bypass cache	+ Avoids cache pollution. – Next read is a miss	Write-once/read-never data (logs)
Refresh-ahead	Pre-fetch cache before expiry (predict usage)	+ Low latency for predicted reads. – Wasteful if prediction wrong	Predictable access patterns

3.2 Cache Invalidation Problems

- **Cache stampede / Thundering herd:** many simultaneous cache misses for same key → all hit DB. Fix: probabilistic early expiration, mutex lock on miss
- **Cache penetration:** query for non-existent key → cache always misses → DB hit every time. Fix: bloom filter, cache null values with short TTL
- **Cache avalanche:** many keys expire simultaneously → DB flooded. Fix: stagger TTL (add random jitter), never-expire + background refresh
- **Data inconsistency:** DB updated but cache stale. Fix: cache invalidation on write, shorter TTL, event-driven invalidation

3.3 Redis vs Memcached

	Redis	Memcached
Data types	String, Hash, List, Set, Sorted Set, Stream, Bitmap, HyperLogLog	String only
Persistence	RDB snapshots + AOF write-ahead log	None — in-memory only
Replication	Master-replica, Sentinel, Cluster	None built-in
Pub/Sub	Yes — basic messaging	No
Lua scripting	Yes — atomic operations	No
Multi-thread	Single-threaded (I/O multi-threaded in Redis 6+)	Multi-threaded — better CPU utilization
Use for	Session store, rate limiting, leaderboards, queues, pub/sub	Simple high-throughput cache — multiple cores

4. Database Selection

DB Type	Examples	Strengths	Weaknesses	Use When
Relational (SQL)	PostgreSQL, MySQL, Oracle	ACID, joins, schema enforcement, mature	Vertical scale, sharding complex	Structured data, transactions, complex queries
Document	MongoDB, CouchDB, Firestore	Flexible schema, JSON-native, scale-out	No joins, eventual consistency	Catalogs, profiles, CMS, nested data
Key-Value	Redis, DynamoDB, Riak	O(1) lookups, massive throughput	No query flexibility	Sessions, cache, shopping cart, counters
Wide-Column	Cassandra, HBase, Bigtable	Linear scale, high write throughput, time-series	No ad-hoc queries, eventual consistency	IoT, logs, metrics, activity feeds
Graph	Neo4j, Amazon Neptune	Relationship traversal, flexible edges	Limited scale, niche	Social networks, recommendations, fraud detection
Time-Series	InfluxDB, TimescaleDB, Prometheus	Optimized for time-range queries, rollups	Single-purpose	Metrics, monitoring, IoT sensor data
Search	Elasticsearch, Solr, OpenSearch	Full-text, faceting, relevance scoring	Eventual consistency, not primary store	Product search, log analysis, autocomplete
NewSQL	CockroachDB, Spanner, TiDB	SQL + horizontal scale + ACID	Complexity, cost	Global distributed ACID transactions

4.1 Database Scaling Patterns

- **Read replicas:** primary handles writes, replicas handle reads. Good for read-heavy (80-90% reads). Replication lag is a tradeoff
- **Sharding (horizontal partitioning):** split data across multiple DB instances by shard key
 - Range sharding: by user ID range — hot spots if skewed
 - Hash sharding: hash(key) % shards — even distribution, range queries harder
 - Directory sharding: lookup table for shard mapping — flexible but lookup overhead
- **CQRS:** separate write model (normalized, transactional) from read model (denormalized, optimized for queries)
- **Polyglot persistence:** different DBs for different needs — RDBMS + Redis cache + Elasticsearch search

5. Asynchronous Design & Messaging

Pattern	Description	Use When
Message Queue	Producer puts work; consumer pulls and processes. Decouple services	Async task execution, rate limiting, background jobs
Pub/Sub	Publisher sends to topic; multiple subscribers receive independently	Event fan-out, notifications, multiple consumers
Event Streaming	Ordered log; replay possible; consumers track offset	Audit log, event sourcing, real-time analytics
Request-Reply async	Send request with correlation ID; consumer replies to reply queue	Async RPC — long-running operations with response needed
Outbox Pattern	Write event to outbox table in same DB transaction as business data; relay publishes	Guarantee event published if DB write succeeds — exactly-once
Saga	Chain of compensating transactions. Choreography (events) or Orchestration (central coordinator)	Distributed transactions across services without 2PC

6. Rate Limiting Algorithms

Algorithm	How It Works	Pros	Cons
Fixed Window	Count requests per fixed time window (per minute). Reset at window end	Simple to implement	Burst at window boundary (double traffic at reset)
Sliding Window Log	Store timestamp of each request. Count in sliding window	Accurate, no boundary issue	Memory: store every timestamp
Sliding Window Counter	Weighted combo of current + previous window count	Memory efficient, accurate approximation	Approximate, not exact
Token Bucket	Bucket refills at fixed rate. Request consumes token. Burst allowed up to bucket size	Allow bursts, smooth average rate	Slightly complex
Leaky Bucket	Requests queue up; processed at fixed rate. Overflow dropped	Smooth output rate	No burst handling, latency for queued requests

✓ Token Bucket: most common. Redis INCR + EXPIRE for distributed rate limiting. Sliding Window: most accurate. Pick based on burst tolerance

7. Consistent Hashing

- Hash ring $0 \rightarrow 2^{32}-1$. Each server = multiple virtual nodes on ring. Key → find next clockwise server

- Adding/removing server: only neighbor keys remapped. Unlike modulo hashing which remaps all keys
- Virtual nodes: distribute load evenly. Server with more capacity gets more virtual nodes
- **Uses:** distributed caches (Redis Cluster), load balancing, distributed DB sharding, CDN routing

8. Classic System Design Examples

8.1 URL Shortener (e.g. bit.ly)

- **Core API:** POST /shorten {url} → short code. GET /{code} → 301/302 redirect
- **Short code generation:** Base62(auto-increment ID) or random 7-char Base62 ($62^7 = 3.5T$ URLs)
- **DB:** KV store (DynamoDB/Redis) — code → long URL mapping. Simple, fast $O(1)$ lookup
- **Scale:** cache hot URLs in Redis (20% of URLs = 80% traffic). CDN for redirect. Read replicas
- **Tradeoffs:** 301 (permanent, client caches, less server load) vs 302 (temporary, every request hits server, enables analytics)

8.2 Design Twitter/X Feed

- **Write:** post tweet → Kafka → fan-out to follower timelines (push model) or fetch on read (pull model)
- **Push (fan-out on write):** pre-compute timeline for each follower in Redis. Fast read $O(1)$. Slow write for celebrities
- **Pull (fan-out on read):** merge recent tweets from followees at read time. Slow read, fast write
- **Hybrid:** push for normal users, pull for celebrity accounts ($>N$ followers)
- **Storage:** tweets in Cassandra (wide-column, high write). User data in MySQL. Media in S3

8.3 Design WhatsApp / Chat

- **Connection:** WebSocket for persistent bidirectional. Long polling fallback
- **Message flow:** sender → server → store in DB → push to recipient via WebSocket or push notification
- **Message storage:** Cassandra — (conversation_id, message_id DESC). Good for append + range queries
- **Presence:** Redis TTL per user. Heartbeat every 5s renews TTL. No key = offline
- **Delivery receipts:** sent/delivered/read status. Delivered = server received. Read = recipient opened

8.4 Design Rate Limiter

- Centralized: Redis + Lua script (atomic). Key = user_id:window. INCR + EXPIRE in one atomic operation
- Distributed: each node has local counter + sync periodically. Approximate but low latency
- **Headers:** X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset, Retry-After
- **Rate limit by:** user ID, IP, API key, endpoint, or combination

8.5 Design a Notification System

- **Channels:** push (APNs/FCM), SMS (Twilio), email (SendGrid/SES), in-app
- **Flow:** service emits event → notification service → prioritize → channel-specific workers → delivery
- **Reliability:** at-least-once delivery. Idempotency key to dedup. Retry with backoff. DLT for failures
- **User prefs:** notification preferences per channel per type. Respect do-not-disturb hours
- **Scale:** Kafka for event ingestion. Worker pools per channel. Exponential backoff on rate limits

9. Key Architecture Patterns

Pattern	Description	Benefit
---------	-------------	---------

API Gateway	Single entry point — auth, rate limit, routing, request aggregation, SSL termination	Decouples clients from services. Cross-cutting concerns in one place
Circuit Breaker	Track failure rate. Trip (open) when threshold exceeded. Half-open to test recovery	Prevent cascade failures. Fast-fail instead of waiting on timeout
Bulkhead	Isolate thread pools/connection pools per service. One slow service doesn't exhaust shared pool	Fault isolation — failure in one silo doesn't affect others
Retry + Backoff	Retry transient failures. Exponential backoff with jitter to avoid thundering herd	Resilience against transient faults. Jitter spreads retry load
Sidecar	Deploy proxy alongside each service (same pod). Handles networking: mTLS, tracing, retries	Service mesh (Istio/Linkerd) — offload cross-cutting from app code
Event Sourcing	Store state as sequence of events. Current state = replay events. Never mutate	Full audit trail, time-travel, replay for new projections
CQRS	Separate command (write) model from query (read) model. Different stores optimized for each	Scale reads and writes independently. Optimized query models
Strangler Fig	Incrementally replace legacy system — new features in new service, route traffic	Safe migration from monolith to microservices
Saga	Distributed transaction via sequence of local transactions with compensating rollback events	Avoid distributed 2PC. Eventual consistency across services

10. Interview Questions

Fundamentals

Q1. What is the CAP theorem?

→ In distributed systems, can guarantee only 2 of: Consistency (all nodes same data), Availability (every request gets response), Partition Tolerance (system works despite network partition). Since partitions always happen, choose CP (reject during partition, e.g., HBase) or AP (return stale data, e.g., Cassandra)

Q2. Explain eventual consistency

→ System guarantees that given no new writes, all replicas eventually converge to same value. Temporary inconsistency possible between replicas. Used by: DynamoDB, Cassandra, DNS. Acceptable for: social feeds, recommendations. Not OK for: balances, inventory

Q3. Horizontal vs vertical scaling?

→ Vertical: add CPU/RAM to one machine — simple, limited, SPOF. Horizontal: add more machines — requires stateless design, LB, distributed coordination. Prefer horizontal for web tier. Vertical for DB until sharding required

Q4. What is consistent hashing?

→ Hash ring for distributing keys to nodes. Adding/removing node: only adjacent keys move ($O(1/N)$ remapped). Without it: modulo hashing remaps all keys on topology change. Used in Redis Cluster, CDN routing, Cassandra partition

Q5. Explain the difference between SQL and NoSQL

→ SQL: ACID, schema enforcement, joins, vertical scale, complex queries (OLTP/OLAP). NoSQL: flexible schema, horizontal scale, various data models (doc/KV/wide-column/graph). NoSQL not always better — choose based on access patterns, consistency needs, scale

Caching & Storage

Q6. What cache strategy do you use and why?

→ Cache-aside (lazy loading): most common. App checks cache, on miss loads from DB and populates cache. Write-through: cache stays fresh but double-write latency. Write-behind: fast writes, risk of data loss. Choice depends on read/write ratio and consistency tolerance

Q7. How to handle cache stampede?

→ Multiple clients simultaneously query cache for expired/missing key, all miss, all hit DB. Fix: (1) Probabilistic early expiration — randomly refresh before expiry. (2) Mutex/distributed lock on miss — first acquirer fetches, others wait. (3) Stale-while-revalidate

Q8. How would you design a globally distributed database?

→ CockroachDB/Spanner approach: shards distributed globally, Paxos/Raft consensus per shard, leader election per region, followers serve local reads. Tradeoff: cross-region write latency (150ms+). For AP: Cassandra multi-datacenter with LOCAL_QUORUM reads

Q9. Explain database sharding

→ Split data across multiple DB instances (shards) by shard key. Hash sharding: even distribution. Range sharding: good for range queries, hot spots. Directory sharding: flexible lookup. Challenges: cross-shard joins, rebalancing, hot spots

Design Problems

Q10. How would you design a URL shortener?

→ API: POST /shorten, GET /{code} → redirect. Code: Base62 encoding of auto-increment ID or random 7 chars. DB: KV store (DynamoDB). Cache: Redis for hot URLs. CDN: geographic redirect. 301 vs 302: 302 for analytics tracking. Estimate: 100M URLs, 10K QPS

Q11. Design a rate limiter

→ Token bucket (allows bursts) or sliding window (accurate). Redis INCR+EXPIRE for distributed. Key: user_id:endpoint>window. Add headers: X-RateLimit-Limit/Remaining/Reset. Tiered limits: free vs paid. Graceful degradation: soft limits before hard cutoff

Q12. Design a notification system

→ Producer services → Kafka → notification workers per channel (push/SMS/email). User preference service. APNs/FCM for mobile push. Retry with exponential backoff. DLT for failed. Dedup with idempotency key. User opt-out/preferences. Respect DND hours

Q13. Design Twitter/X news feed

→ Hybrid fan-out: push for regular users (pre-computed Redis timelines), pull for celebrities. Store tweets in Cassandra (high write). Media in S3 + CDN. Timeline reads from Redis sorted set. Pagination by tweet_id. Follow graph in separate graph service

Q14. How do you design for high availability?

→ Redundancy: no SPOF. Load balancing: distribute across instances. Health checks + auto-replace. Multi-AZ deployment. Circuit breaker: fail fast. Graceful degradation: serve stale data over errors. Chaos engineering: test failure scenarios

Q15. What is the difference between a message queue and a pub/sub system?

→ Queue: each message consumed by one consumer. Load distributed. Message deleted after processing. Pub/Sub: message broadcast to all subscribers. Each subscriber gets full copy. Use queue for task distribution, pub/sub for event notification to multiple systems

Patterns & Trade-offs

Q16. Explain circuit breaker pattern

→ 3 states: Closed (normal — requests pass through), Open (failure threshold exceeded — all requests fail fast), Half-open (test if downstream recovered — one request allowed). Prevents cascade failures. Libraries: Resilience4j, Hystrix (deprecated)

Q17. What is the outbox pattern?

→ Solve dual-write problem: writing to DB + publishing to message queue atomically. Solution: write business data + event to outbox table in same transaction. Separate relay/CDC reads outbox and publishes to Kafka. Guarantees event published if DB write succeeds

Q18. Explain event sourcing

→ Store state as append-only sequence of events. Current state = apply all events from beginning. Benefits: full audit log, replay events for new projections, time travel. Challenges: event schema evolution, read performance (snapshot + recent events)

Q19. What is CQRS?

→ Command Query Responsibility Segregation: separate write model (normalized, transactional DB) from read model (denormalized, read-optimized store). Commands change state, queries read state. Often combined with event sourcing. Trade-off: eventual consistency between models

Q20. How do you handle distributed transactions without 2PC?

→ Saga pattern: sequence of local transactions with compensating transactions for rollback. Choreography: services emit events and react. Orchestration: central saga coordinator. Outbox pattern ensures at-least-once event delivery. Accept eventual consistency